

公司 HR 制度智能问答项目：人机交互对话提炼

来源线程：codex://threads/01a010a0-1a62-77c3-b092-dc2a9c02609f（需求分析）、codex://threads/01a041f9-ef7b-77c1-a7f0-4c5852bed64e（模块化开发）

整理日期：2026-08-31

说明：本文按项目决策链提炼主要对话。为便于阅读，内容以忠实转述为主，不等同于逐字聊天记录；测试数字、命令状态和任务完成度均按线程中可见证据重新审计。

一、项目对话主线

项目从“基于知识库检索生成的企业 HR 制度问答助手”出发，经过需求澄清、范围收敛、创新设计、技术栈调整、模块化开发和多层验证，逐步形成如下方案：

- 员工端：自然语言问答、多轮追问、低证据拒答、会话历史、逐结论证据链和制度原文定位。
- HR 端：管理员登录、制度版本维护、文件解析、索引重建、检索调试及后续意见闭环。
- RAG 核心：BGE 中文向量检索、BM25、RRF 融合排序，DeepSeek 仅负责基于证据组织结构化回答。
- 创新能力：情景决策沙盘、一问一办、答案保鲜、制度共创意见箱及数据洞察。
- 明确边界：不建设真实员工档案、考勤审批、自动算薪、复杂企业权限和生产级部署。

对话的价值不只在“AI 生成了代码”，更在于人持续修正范围、技术路线和验收口径，AI 则承担资料核验、实现、测试与交接记录。最终形成的是“人定目标和边界—AI 提出方案并执行—人纠偏—工具验证—共同封板”的协作链路。

二、主要对话提炼

第 1 轮：从题目描述到可验收系统

人：项目选择“公司 HR

制度智能问答”，要求知识库检索生成、自然语言问答、制度条款精准定位、多轮对话和溯源引用。

AI：建议新建独立 Flask 项目，使用虚构 HR 制度、本地 BGE 向量、BM25 与 RRF 混合检索，在线模型负责问题改写和答案组织；API 异常时只返回本地条款。

批判性审查：AI 最初把范围默认成“员工端、无复杂后台”，这是合理的 MVP 假设，但不是用户已确认的事实。用户随后要求同时考虑 HR 管理端，说明 AI 的首次范围判断需要通过追问而不是直接固化。

协作结果：把模糊题目转化为可测指标：25 条评测问题、Recall@3 ≥ 85%、无依据问题拒答、引用可定位、会话可持久化。

第 2 轮：课程时间线改变项目管理方式

人：提供课程时间线，并补充七类交付物：立项报告、源代码、用户手册、展示 PPT、项目视频、关闭总结报告和个人总结。

AI：从课程材料中识别到“发现并纠正 AI 错误、验证过程和人机分工”本身是评分点，因此把 AI 协作证据加入每日产出，并规划需求、研发、初验、迭代、终验和复盘。

批判性审查：AI 曾按图片推断终验为 9 月 4 日，用户后续明确更正为 9 月 3 日。日期属于硬约束，必须以用户最新口径覆盖旧计划，而不能沿用早期推断。

协作结果：项目计划从“只完成软件”升级为“软件、测试证据、文档和答辩材料同步推进”。

第 3 轮：用结构化澄清抑制范围漂移

人：安装 requirements-clarity 技能并开展需求调研；随后确认员工端和 HR 端都做、允许虚构制度、现场可联网。

AI：按 100 分制记录需求清晰度，从 86 分提升到 89 分，并追问 HR 端究竟是知识库维护，还是包含员工档案、考勤、薪资等 HRIS 功能。

人：一度提出同时加入员工档案、考勤审批和薪资管理，并询问能否直接调用 GPT。

AI：指出项目已接近小型 HRIS，在 9 月 3 日终验约束下会显著增加权限、隐私、状态流转和测试成本；同时说明 ChatGPT 会话不能被 Flask 当作 API，必须使用独立 API key，且密钥只能放在后端环境变量中。

批判性审查：这里没有把“功能越多”当成“需求更清晰”。范围扩大后，AI 将清晰度从 89 降到 84，说明它识别了未定义的数据字段、权限和业务闭环。最终报告又删除 HRIS 模块，使产品主线回到制度问答与知识治理。

协作结果：人提出业务愿望，AI 量化进度与复杂度风险，双方最终以 KISS/YAGNI 原则冻结边界。

第 4 轮：模型选择不是事实来源选择

人：质疑“为什么要接 DeepSeek”，并询问能否直接调用 GPT；后续开发阶段又明确改用 DeepSeek。

AI：解释本地检索负责找到制度事实，大模型只负责理解追问和组织语言；模型应通过适配器替换，失败时降级为本地证据。

多角度验证：

- 产品角度：自然问答与多轮追问需要语言模型，但条款检索本身不依赖模型生成。
- 安全角度：浏览器不能保存 API key；模型输出必须当作不可信数据。
- 稳定性角度：无密钥、超时、限流或非法结构时，不生成制度结论。
- 可测试性角度：自动测试使用确定性模型替身，在线接口只做显式冒烟测试。

协作结果：模型厂商从 OpenAI 候选切换为 DeepSeek，但“检索决定事实、模型组织表达、服务端校验证据”的架构原则保持不变。

第 5 轮：从基础 RAG 到创新闭环

人：要求完善创新点，并同步修改正式 Markdown 和 PDF。

AI：将需求从普通问答扩展为：情景条件澄清、条件切换推演、逐结论可点击证据链、一问一办、知识指纹与答案保鲜、意见关联与回归复测。

批判性审查：创新功能不能只写在概述里。AI 同步修改需求编号、角色占比、优先级统计、竞品矩阵、风险、功能矩阵、业务流程和验收场景，避免“正文说有创新，追踪矩阵却没有”的前后矛盾。

能力边界：情景沙盘只解释制度适用条件，不替代正式审批或法律判断；意见箱只形成知识修复闭环，不等同于企业级工单平台。

第 6 轮：技术栈与运行方式由人最终拍板

AI 初始建议：Flask 单体应用，Jinja + 原生 JavaScript；开发时前后端分开，最终可由 Flask 托管 Vue 静态资源。

人：明确采用 Vue 3 + Flask + SQLAlchemy + SQLite，并要求 F01-F18 全覆盖；随后要求多任务模块化开发，最终仍按传统方式让前后端分别运行。

AI 修正：改用 Vue 3、Vite、TypeScript strict、Pinia、Vue Router、Element Plus；后端采用 Flask 应用工厂、SQLAlchemy、Flask-Migrate、SQLite；最终保持 5000/5173 两个服务，并通过 CORS、Cookie 和 CSRF 串接。

发现并纠正 AI 错误：Jinja 方案和“最终合并为单服务”的建议都被用户明确否决。AI 没有把先前方案包装成最终决定，而是更新正式报告、接口契约和启动方式。

协作结果：
人负责产品形态、技术栈和演示习惯；AI 负责把选择转译为目录结构、接口规范、安全配置和任务交接规则。

第 7 轮：模块化开发中的分工与验证

任务 1——工程骨架与契约冻结

- **人：** 决定多任务模块化开发，要求最终串联。
- **AI：** 建立 Vue/Flask 骨架、数据库模型、迁移、CORS、CSRF、统一错误格式、API 契约、数据库设计和 F01-F18 追踪矩阵。
- **验证：** 后端 4 项测试、前端 1 项测试、TypeScript 检查和 Vite 构建通过；桌面与 390 px 布局得到检查。

任务 2——知识库与混合检索

- **人：** 要求继续完成任务 2。
- **AI：** 实现四种文档解析、版本规则、稳定锚点、知识指纹、BGE + BM25 + RRF、原子索引和 Top 5 调试。
- **验证：** 30 条示例条款、25/25 命中，Recall@3 = 100%；后端 13 项测试、前端检查及真实 HTTP 阅读器冒烟通过。
- **边界分析：** 100% 只说明自建 25 题评测集全部命中，不能外推为真实企业问题的普适准确率。

任务 3——可信问答与会话

- **人：** 要求基于交接记录完成任务 3。
- **AI：** 实现最近 6 轮上下文、低分拒答、DeepSeek JSON Schema、声明—证据映射、数字一致性检查、会话隔离和证据快照。
- **验证：** 后端 22 项测试；在线冒烟得到 2 条声明、2 条证据、覆盖率 1.0；“宠物补贴”在模型调用前被拒答；预热检索约 0.024 秒。
- **边界分析：** 单次在线冒烟证明链路可用，不证明第三方 API 的长期稳定性；0.024 秒是预热后的检索耗时，不是端到端回答延迟。

任务 4——前端整合

- **人：** 要求在各阶段交接记录基础上完成任务 4。
- **AI：** 接入员工会话、多轮消息、拒答/降级/过期提示、逐声明证据按钮、条款高亮，以及 HR 登录、制度预览、索引和 Top 5 诊断。
- **验证：** TypeScript、构建和组件测试通过；浏览器实测员工问答、证据高亮、多轮追问、HR 登录、制度预览、Top 5、退出和 390 px 响应式布局。
- **状态审计：** 该轮线程最终状态为 interrupted，虽有大量已执行证据，但没有完整最终交付答复，因此只能认定“主体实现和浏览器验收已发生”，不能直接认定任务 4 已完整封板。

第 8 轮：测试失败不是项目失败，而是进一步调查入口

浏览器验收中出现多次失败：SPA 导航等待失败、Top 5 首次点击未得到预期状态、若干浏览器驱动调用方法不匹配。AI随后读取页面状态、换用实际可用定位方法并重试，最终完成相应检查。

这体现了有效的批判性循环：

提出假设 → 执行测试 → 观察失败 → 区分产品缺陷与测试脚本缺陷 → 调整方法 → 再验证

其中，导航失败主要是测试对 SPA 行为的假设不合适，并不能直接判定页面不可用；只有检查实际 URL、DOM 和控制台日志后，才可归因。

第 9 轮：从“复述制度”改为“先给明确结论”

用户先要求完整审计员工问答链路，并特别检查 Prompt、结构化输出、员工档案、办理助手、流程来源和制度模型；随后明确提出：对于可判断的问题，回答必须优先给出“可以、不可以、需要、不需要、符合、不符合、条件不足，暂时无法判断”之一，不能只复述制度。

审计确认原链路虽有检索、证据校验和澄清机制，但没有稳定的业务结论字段，且部分场景会在制度已经足以否定时继续追问条件。AI据此完成以下最小改造：

- 后端结构化输出增加 decision = allowed / denied / conditional / informational、明确结论、下一步和缺失条件字段。
- 回答顺序固定为“明确结论 → 结合当前员工情况解释原因 → 下一步怎么办 → 制度依据”。
- 检索和模型判断先于条件追问；如果制度已足以判断“不可以”，立即结束无意义的补充条件流程。
- 信息不足时返回 conditional，并列出的必要条件；不向员工展示隐藏推理过程，只展示可核验规则和制度证据。
- 前端同步展示结论、原因、缺失条件、下一步和依据；非 conditional 回答不再显示条件补充表单。
- 补充“试用期年假”“漏打卡补卡”“差旅报销”三类自动化测试，并同步数据库迁移、API 契约和演示聊天记录。

本轮验证结果为：后端 45 项测试通过，前端 17 项测试通过，前端生产构建通过，数据库迁移可从基础版本完整升级到最新版本。该轮也纠正了旧流程中“先按静态槽位追问、再交给模型判断”的顺序问题。

三、批判性思维证据

1. 发现并纠正 AI 错误

编号	AI 初始判断或输出	人/证据发现的问题	纠正后的结论
C1	初期默认不做 HR 管理后台	用户要求员工端和 HR 端都做	保留轻量知识治理后台，但排除完整 HRIS
C2	时间线按 9 月 4 日终验规划	用户明确终验是 9 月 3 日	所有里程碑以 9 月 3 日为硬截止
C3	提议 Jinja + 原生 JS	用户明确指定 Vue 3 技术栈	改为 Vue 3 + Vite + TypeScript

编号	AI 初始判断或输出	人/证据发现的问题	纠正后的结论
C4	建议最终前后端合并单服务	用户要求传统前后端分别运行	固定 5000/5173 双服务及 CORS/CSRF
C5	一度把 F18 视为 P2、本期不做	用户强调 F01-F18 全覆盖	F18 纳入后续任务和追踪矩阵
C6	最终总结称传统启动通过	日志中新的 Waitress/python run.py 启动命令退出码为 1，但已有 HTTP 服务可访问	只能证明当时接口可访问；独立冷启动仍需清理端口后复测
C7	任务 4 接近完成并准备封板	线程状态实际为 interrupted	标为“主体实现和多项验收已完成，最终封板证据不完整”

2. 多角度验证

验证层	方法	解决的问题	局限
需求层	清晰度评分、KISS/YAGNI、F01-F18 追踪	防止范围漂移和遗漏	分数是项目内方法，不是客观行业标准
契约层	API 文档、数据库设计、前端类型同步	防止模块间字段不一致	文档一致不代表运行时一定正确
单元/组件层	pytest、Vitest、TypeScript strict	验证局部逻辑和类型	不能覆盖真实浏览器与第三方 API
检索层	25 题 Recall@3、Top 5 分数查看	验证条款能否被召回	自建样本规模小，存在过拟合风险
模型层	JSON Schema、伪造证据、数字一致性、低分拒答	限制幻觉和错误引用	语义一致性校验仍可能误判
在线层	DeepSeek 单次真实冒烟	证明真实接口链路可用	不代表长期稳定、成本和限流表现
端到端层	浏览器 DOM、控制台日志、桌面/移动视口	验证真实交互和布局	任务 4 未形成完整最终封板记录
文档层	Markdown/PDF 同步、逐页排版检查	避免需求数字和版面冲突	文档正确不等于功能已全部实现

3. 能力边界分析

- **事实边界：** 大模型不决定制度事实；只有当前启用版本中的检索证据可以支撑结论。
- **权限边界：** 当前是课程演示系统，不具备企业 SSO、细粒度 RBAC、审计合规和生产级密钥治理。
- **数据边界：** 使用虚构制度和匿名会话，不处理真实员工隐私、薪酬或考勤数据。
- **业务边界：** “一问一办”提供办理提示，不提交审批；情景推演不替代 HR 正式解释或法律意见。
- **可靠性边界：** 第三方模型可能超时、限流或输出非法结构，因此必须保留拒答和本地证据降级。
- **评测边界：** 25 题 100% Recall@3 是内部数据集结果，不能等价为真实环境 100% 准确。
- **状态边界：** 命令失败、线程中断和工具重试必须在总结中保留，不能被“最终成功”叙述抹平。
- **AI 能力边界：**
AI 适合生成候选方案、代码、测试和文档，但无法代替用户决定课程范围，也不能仅凭自述证明功能完成。

四、人机协作流程

1. 标准协作闭环

人提出目标/纠偏 → AI 检索上下文并给出候选方案 → 人选择范围与技术路线 → AI 实施 → 自动化与浏览器验证 → 人或证据发现问题 → AI 修正 → 更新契约与交接记录

2. 有效迭代轮次

下表按“产生了可验证决策或交付物”统计有效迭代，而不是按消息数量机械计数。

有效轮次	核心问题	人的贡献	AI 的贡献	可验证产出
I1	题目如何落地	给出选题与必备能力	形成 RAG、引用、会话和评测方案	初始实施计划
I2	如何对齐课程	提供时间线和交付物	加入 AI 纠错证据链与里程碑	完整项目排期
I3	范围是否膨胀	提出员工端、HR 端及 HRIS 想法	量化风险并推动范围收敛	需求调研记录、PRD
I4	创新如何体现	要求创新点与正式文档同步	设计沙盘、证据链、保鲜、意见闭环	F01-F18 功能矩阵
I5	技术栈与运行方式	指定 Vue 3、模块化、传统双服务	更新架构、契约和任务拆分	开发计划与交接规范
I6	工程基础	批准按任务执行	完成骨架、数据库和契约	任务 1 代码与测试
I7	检索可信度	要求继续任务 2	完成知识库与真实 BGE 评测	13 项测试、25/25 评测
I8	生成可信度	要求完成任务 3	完成拒答、降级和证据校验	22 项测试、在线冒烟
I9	前端真实可用性	要求完成任务 4	完成页面串接与浏览器测试	组件测试、浏览器验收记录
I10	证据审计	要求提炼两线程	区分已验证事实、失败日志和未封板状态	本文及对应 PDF
I11	回答必须先下结论	要求审计链路并定义七类结论	增加决策契约、调整生成顺序、同步前端与测试	45 项后端测试、17 项前端测试、迁移与构建通过

3. 人机分工标注

工作类型	人主责	AI 主责	共同负责
项目目标	选择题目、确定课程优先级	把目标转成可执行规格	校准成功标准
范围决策	决定做/不做及硬截止	提示复杂度、隐私和进度风险	冻结 F01-F18 与非范围
技术路线	最终选择 Vue/Flask/SQLite/DeepSeek	比较方案、核验官方接口、搭建架构	形成可替换和可降级设计
开发实现	审阅方向、发起阶段任务	编码、迁移、接口、类型和文档	处理公共契约变化
质量验证	判断验收是否符合课程目标	运行单测、评测、冒烟和浏览器检查	对失败进行归因和复测
风险承担	决定是否接受范围、成本和交付风险	识别但不能替用户接受风险	记录限制与遗留项
最终结论	人工批准与答辩	提供证据化总结	不把 AI 自述当作唯一证据

五、对协作质量的评价

做得较好的方面

- 对需求范围进行多轮收敛，没有把完整 HRIS 混入制度问答主线。
- 模型输出被设计成“不可信输入”，必须经过证据 ID、版本、数字和相关度校验。
- 使用契约、迁移、类型、单元测试、真实模型、HTTP 和浏览器多层验证，而非只看代码能否编译。
- 每个开发任务都读取上阶段交接记录，减少多任务开发中的接口漂移。
- 用户多次推翻 AI 的默认建议，最终技术栈和运行方式以用户决策为准。

仍需改进的方面

- 计划在“单体/前后端分离”“OpenAI/DeepSeek”“F18 是否本期实现”之间多次切换，说明早期决策表应更快形成并版本化。
- AI 的最终总结有时弱化了失败命令。例如已有服务可访问时，新启动进程因端口占用失败，不能直接写成“启动通过”。
- 任务 4 的浏览器验收工具调用有多次方法错误，应在首次失败后更快回到已加载的工具文档和已验证调用范式。
- 检索评测集由项目自身设计，后续应增加教师问题、同学盲测和对抗性问题，降低自测偏差。
- 需求全覆盖与短工期存在张力。创新功能应继续按 P0/P1 分层，避免“矩阵中全覆盖”被误解成每项都达到生产质量。

六、最终结论

两条线程体现了较完整的人机协作过程：人不是被动接受 AI 输出，而是持续纠正日期、范围、技术栈、运行方式和功能优先级；AI 也不仅生成方案，而是把决策落实为契约、代码、测试、浏览器验收和交接记录。

最具批判性价值的不是“测试全部通过”这类结论，而是对证据强度的分级：内部评测不能代表真实泛化，单次在线冒烟不能代表长期稳定，已有 HTTP 服务可访问不能替代独立冷启动验证，线程中断也不能被写成完整封板。保留这些限制，使项目答辩中的 AI 协作叙述更可信、更可复核。

建议用于答辩的概括：人负责目标、边界、取舍和最终验收；AI 负责信息整理、方案候选、代码实现和自动化验证；双方通过“提出—质疑—验证—修正—留痕”的循环共同完成项目。